

Building Northwind Trading on aspgen

A complete, production-oriented walkthrough of every architecture tier, UI framework, and flag aspgen supports — built around one real, multi-context enterprise solution.

DOCUMENT	AUDIENCE	STATUS	THEME
Enterprise Developer Guide	.NET engineers adopting aspgen	v1.0, 2026-08-05	aspgen Document Theme

DECISION aspgen generates ASP.NET Core Clean Architecture APIs and Prism/Dryloc WPF apps from embedded Go `text/template` sources — nothing is downloaded at generation time and nothing runs as a service. The recommended entry point is the **context/arch engine** (`--context` / `--arch` / `-ui`), which lets one solution mix independent bounded contexts at four architecture tiers (`ar` , `dm` , `cqrs` , `es`) and attach one UI framework (`wpf` , `blazor` , `mvc` , or `spa`) on top. An older, still fully supported `--app` / `--backend` **workflow** (including the Renoir-style DDD Blazor profile) remains available for teams already using it. This guide builds a real multi-context system — **Northwind Trading** — end to end on the modern engine, then tours the legacy workflow and every flag aspgen accepts.

Contents

1. What you'll build
2. How aspgen thinks: two workflows, one generator

3. Install and verify aspgen
4. The architecture ladder — `ar` → `dm` → `cqrs` → `es`
5. Scaffolding the solution shell
6. Context 1 — Catalog (`ar`): a lean, headless product API
7. Context 2 — Inventory (`dm`): aggregates, value objects, domain services, repositories, events
8. Context 3 — Sales (`cqrs`): vertical-slice commands/queries and a WPF back office
9. Context 4 — Billing (`es`): event sourcing and an append-only ledger
10. A tour of every UI framework (`wpf` , `blazor` , `mvc` , `spa`)
11. Extending every layer — recipes for real feature work
12. The legacy `--app` / `--backend` workflow, including the Renoir DDD Blazor profile
13. Custom templates
14. Testing, CI, and production readiness
15. Full flag reference
16. Troubleshooting and known gotchas
17. Next steps

1. What you'll build

Northwind Trading is a fictional wholesale distributor with four bounded contexts, each modeled at the architecture tier that actually fits its complexity — not the tier that looks the most impressive. The contexts split across **three separate solutions**, because aspgen currently scaffolds each solution's shared Domain/Application/Infrastructure skeleton from whichever context is created first, and that skeleton isn't interchangeable across every tier (see the callout in Section 4 and the Troubleshooting section):

Context	Business need	Tier	Solution	Why this shape
Catalog	Public product lookup, high read volume, few business rules	<code>ar</code>	<code>CatalogApi</code> (standalone)	Flat CRUD is the right amount of ceremony for "here is a product record," and <code>ar</code> 's lean single-project skeleton can't host

Context	Business need	Tier	Solution	Why this shape
				<code>dm / cqrs / es</code> contexts alongside it.
Inventory	Stock levels, reservations, warehouse transfers — invariants matter	<code>dm</code>	<code>NorthwindOps</code> (combined)	Needs an aggregate root protecting invariants; its needs are a subset of <code>cqrs</code> 's skeleton, so it shares a solution with Sales.
Sales	Order capture, pricing, fulfillment — multiple verbs, a real API surface	<code>cqrs</code>	<code>NorthwindOps</code> (combined)	Vertical-slice commands/queries plus a WebApi host — bootstraps the shared skeleton Inventory then reuses.
Billing	Invoices and ledger entries — full auditability, "what happened and when"	<code>es</code>	<code>BillingLedger</code> (standalone)	Event sourcing gives a true audit trail, but <code>es</code> 's event-sourced base class isn't compatible with <code>dm / cqrs</code> 's <code>BaseEntity</code> skeleton, so it gets its own solution.

```

flowchart LR
    subgraph CatalogApi["CatalogApi.sln"]
        Catalog["Catalog\n(ar)"]
    end
    subgraph NorthwindOps["NorthwindOps.sln"]
        Inventory["Inventory\n(dm)"]
        Sales["Sales\n(cqrs)"]
    end
    subgraph BillingLedger["BillingLedger.sln"]
        Billing["Billing\n(es)"]
    end
    Desktop["Desktop\nPrism/DryIoc WPF shell"]
    Storefront["External storefront\n(not generated)"]

    Storefront -->|REST| Catalog
    Desktop -->|in-process CrudService| Inventory
    Desktop -->|HTTP| Sales

```

`NorthwindOps` attaches one `wpf` UI spanning its two contexts, giving warehouse and sales staff a single Prism/Dryloc desktop shell with a module per context — `wpf` is the UI framework most tolerant of mixed tiers within one project (it works against `dm`, `cqrs`, or `es` individually, and against a `dm + cqrs` combination specifically, since `cqrs`'s skeleton is a superset of what `dm` needs). `CatalogApi` stays headless: `ar`-tier contexts get a real Minimal API host but no aspgen-generated UI screens today, which is the correct trade-off for a lean, high-traffic read API meant to be called directly (curl, Postman, a separate storefront app, or a hand-written SPA).

`BillingLedger` stands alone as its own deployable service, given its own UI in Section 10.

Section 10 then tours all four UI frameworks (`wpf`, `blazor`, `mvc`, `spa`), each attached to whichever project actually supports it, since a single project only gets one attached UI.

2. How aspgen thinks: two workflows, one generator

aspgen is a two-step generator, always:

1. `new` initializes a project once — tree, `.sln`, and `.aspgen/manifest.json`.
2. `add` (and `import-db`) mutate that already-generated project incrementally. `add` never creates a missing project; if `--project` is omitted it searches the current directory and its parents for `.aspgen/manifest.json`.

Everything is rendered from Go `text/template` sources embedded in the `aspgen` binary (`internal/templates/files/**`) — export and inspect them any time with `aspgen templates export`.

Two independent flag surfaces select the generation profile:

Workflow	Trigger flag	Status
Context/arch engine	<code>--context NAME --arch ar\ dm\ cqrs\ es</code>	Recommended for all new projects.
Legacy <code>--app / --backend</code>	<code>--app webapi\ wpf\ blazor\ fullstack</code>	Fully supported, no longer the advertised default.

The two are mutually exclusive per project — pick one when you run `new . --context` on the command line is what routes `new / add` into the newer engine; a bare `--app` (or no flags at all, which defaults to `--app webapi`) stays on the legacy path.

Callout — flag forms. Every aspgen flag accepts three equivalent forms: `--flag value` , `--flag:value` , and `-flag:value` . This guide uses the space form throughout; use whichever reads best in your own scripts.

3. Install and verify aspgen

```
git clone <this-repo>
cd aspgen
go build ./cmd/... ./internal/...
go run ./cmd/aspgen version
go run ./cmd/aspgen --help
```

`go run ./cmd/aspgen` is used for every command in this guide; substitute a built `aspgen.exe` if you prefer. aspgen never shells out to `dotnet` itself — you build and test the generated solution with your normal .NET tooling.

4. The architecture ladder — `ar` → `dm` → `cqrs` → `es`

The four tiers are an ordinal ladder: each is a strict superset of the concepts in the tier before it.

Tier	Adds on top of the previous tier	Host project	add kinds available
<code>ar</code>	Flat entity, direct <code>DbContext</code> CRUD	WebApi (Minimal API)	<code>add entity</code>

Tier	Adds on top of the previous tier	Host project	add kinds available
dm	Aggregate root, value objects, domain services, repository (interface + EF impl), event record scaffolds, synchronous <code>CrudService</code>	(none — class libraries only)	add aggregate , add value-object , add domain-service , add repository , add event
cqrs	Vertical-slice Application layer: Command/Query + Handler + FluentValidation validator per verb	WebApi (Minimal API)	all of dm 's kinds, add repository registers with DI
es	Append-only event store, event-sourced aggregates (<code>Raise</code> / <code>ApplyHistoric</code> / <code>LoadFromHistory</code>), synchronous read-model projections	WebApi (Minimal API)	all of dm 's kinds except add repository (already generated)

ar	Active Record: flat entity, direct DbContext CRUD.
dm	Domain Model: aggregate root + value objects + domain services + repository + event record scaffolds + synchronous Application-layer CRUD service. No host project (class libraries only).
cqrs	dm's Domain layer + vertical-slice Application (Command/Query/Handler/Validator per verb) + a WebApi Minimal API host.
es	cqrs tier + an append-only event store, event-sourced aggregates (rehydrated by replaying events), and read-model projections.

A single solution may freely mix tiers across contexts — that's the point of the engine, and exactly how Northwind Trading is built below.

Callout — mixing tiers in one solution. Each solution's shared Domain/Application/Infrastructure/Persistence project skeleton is created once, by whichever context is created first via `new` ; a later `add context --arch X` records the new context's metadata but does not backfill skeleton files the first context's tier never rendered. In practice: a `dm` context can always be added to a project whose first context

is `cqrs` (or another `dm`), since `dm`'s needs are a strict subset of `cqrs`'s. `es` uses a different aggregate base (`EventSourcedAggregate`, not `BaseEntity`) and needs its own event-store plumbing, and `ar` uses an entirely different, simpler single-project skeleton with no separate Domain/Application layers at all — mixing either of those with a different first tier in the same solution isn't reliably supported yet. Give `ar`-tier and `es`-tier contexts their own solution unless they are the very first (and, for `ar`, only) context. See Troubleshooting for the exact symptoms.

5. Scaffolding the solution shell

Bootstrap `NorthwindOps` with Sales at the `cqrs` tier first — `cqrs`'s skeleton is the one that Inventory's `dm` context will reuse:

```
go run ./cmd/aspgen new NorthwindOps `
  --context Sales --arch cqrs `
  --database sqlite `
  --output ./NorthwindOps
```

This single command creates `NorthwindOps.sln`, the Sales context's Domain/Application/Infrastructure/Persistence/WebApi tree at the `cqrs` tier, `tests\NorthwindOps.UnitTests` / `tests\NorthwindOps.IntegrationTests` (pass `--no-tests` to skip both), `scripts\ci.ps1`, and `.aspgen/manifest.json`.

```
NorthwindOps/
├─ NorthwindOps.sln
├─ .aspgen/manifest.json
├─ scripts/
│   └─ ci.ps1
├─ src/
│   ├─ NorthwindOps.DomainModel/
│   │   └─ Sales/
│   ├─ NorthwindOps.Application/
│   ├─ NorthwindOps.Infrastructure/
│   ├─ NorthwindOps.Persistence/
│   ├─ NorthwindOps.Resources/
│   └─ WebApi/
```

```

|   |   | Program.cs
|   |   | Features/Sales/
|   |   |
|   |   | tests/
|   |   |   | NorthwindOps.UnitTests/
|   |   |   | NorthwindOps.IntegrationTests/

```

Add Inventory as a `dm`-tier context on top of that same skeleton — this direction works because `dm` needs strictly less than `cqrs` already provides (no separate host, no vertical-slice Application layer):

```
go run ./cmd/aspgen add context Inventory --arch dm --project ./NorthwindOps
```

Catalog and Billing get their own solutions instead of joining `NorthwindOps` (see the Section 4 callout for why):

```
go run ./cmd/aspgen new CatalogApi --context Catalog --arch ar --database sqlite --out
go run ./cmd/aspgen new BillingLedger --context Billing --arch es --database sqlite --
```

6. Context 1 — Catalog (`ar`): a lean, headless product API

Catalog holds `Product` and `Category`, with a many-to-one relation from product to category:

```
go run ./cmd/aspgen add entity Category name:string --context Catalog --project ./Catalog
go run ./cmd/aspgen add entity Product name:string sku:string price:decimal active:bool
```

`category:Category` synthesizes a required `CategoryId` foreign key and a `Category` navigation property (`nav:Entity?` would make it optional). Property types accepted by `name:type` are `string`, `int`, `long`, `decimal`, `float`, `bool`, `date` (`DateOnly`), `datetime` (`DateTime`), and `guid` / `uuid`; append `?` for nullable (`middleName:string?`). Unknown types are a hard error, not a silent pass-through.

`ar`-tier endpoints are plain Minimal APIs against `DbContext`, including list, paged search, get-by-id, create, update, and delete:


```
// src/WebApi/Features/Catalog/Product/ProductEndpoints.cs (rendered)
public static class ProductEndpoints
{
    public static void MapProductEndpoints(this WebApplication app)
    {
        var group = app.MapGroup("/api/catalog/product");
        group.MapGet("/", async (AppDbContext db, CancellationToken cancellationToken)
            Results.Ok(await db.Products.AsNoTracking().ToListAsync(cancellationToken));
        group.MapGet("/search", async (string? search, decimal? priceMin, decimal? priceMax, bool active,
            int page, int pageSize, AppDbContext db, CancellationToken cancellationToken)
        {
            if (page < 1) page = 1;
            if (pageSize < 1) pageSize = 25;
            var query = db.Products.AsNoTracking().AsQueryable();
            if (!string.IsNullOrEmpty(search)) query = query.Where(x => x.Name.Contains(search));
            if (priceMin.HasValue) query = query.Where(x => x.Price >= priceMin.Value);
            if (priceMax.HasValue) query = query.Where(x => x.Price <= priceMax.Value);
            if (active.HasValue) query = query.Where(x => x.Active == active.Value);
            var totalCount = await query.LongCountAsync(cancellationToken);
            var items = await query.OrderBy(x => x.Id).Skip((page - 1) * pageSize).Take(pageSize).ToListAsync(cancellationToken);
            return Results.Ok(new { items, totalCount, page, pageSize, hasMore = (totalCount > (page * pageSize)) });
        });
        group.MapGet("/{id:long}", async (long id, AppDbContext db, CancellationToken cancellationToken)
            Results.Ok(await db.Products.FindAsync(id)));
        group.MapPost("/", async (Product request, AppDbContext db, CancellationToken cancellationToken)
            Results.Ok(await db.Products.AddAsync(request)));
        group.MapPut("/{id:long}", async (long id, Product request, AppDbContext db, CancellationToken cancellationToken)
            Results.Ok(await db.Products.UpdateAsync(request)));
        group.MapDelete("/{id:long}", async (long id, AppDbContext db, CancellationToken cancellationToken)
            Results.Ok(await db.Products.RemoveAsync(id)));
    }
}
```

Because Catalog has no attached UI screens, this is a real, callable REST API today: run the WebApi host and hit `/api/catalog/product/search?search=widget&page=1&pageSize=25`, or point OpenAPI/Scalar at it (`/openapi/v1.json`, `/scalar/v1`).

Seeding Catalog with real Northwind sample data

The legacy `--app / --backend` workflow has a `--seed dummy` flag (Section 12), but it only generates generic, type-based placeholder values (`"Name sample 1"`, incrementing numbers) — and it isn't available on the context/arch engine at all yet. For a demo that actually looks like a wholesale distributor, hand-add a small idempotent seeding block at the `// aspgen:seed`

marker `new / add entity` already left in `Program.cs`, using the real classic Northwind categories and products (the same public sample dataset Microsoft has shipped in Access/SQL Server tutorials for decades):

```
// src/WebApi/Program.cs – add right after "// aspgen:seed"
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
    if (!db.Categorys.Any())
    {
        var categories = new[]
        {
            new Category { Name = "Beverages" },
            new Category { Name = "Condiments" },
            new Category { Name = "Confections" },
            new Category { Name = "Dairy Products" },
            new Category { Name = "Grains/Cereals" },
            new Category { Name = "Meat/Poultry" },
            new Category { Name = "Produce" },
            new Category { Name = "Seafood" },
        };
        db.Categorys.AddRange(categories);
        db.SaveChanges();

        db.Products.AddRange(
            new Product { Name = "Chai", Sku = "BEV-001", Price = 18.00m, Active = true },
            new Product { Name = "Chang", Sku = "BEV-002", Price = 19.00m, Active = true },
            new Product { Name = "Aniseed Syrup", Sku = "CON-001", Price = 10.00m, Active = true },
            new Product { Name = "Chef Anton's Cajun Seasoning", Sku = "CON-002", Price = 22.00m, Active = true },
            new Product { Name = "Pavlova", Sku = "SWT-001", Price = 17.45m, Active = true },
            new Product { Name = "Teatime Chocolate Biscuits", Sku = "SWT-002", Price = 12.99m, Active = true },
            new Product { Name = "Queso Cabrales", Sku = "DAI-001", Price = 21.00m, Active = true },
            new Product { Name = "Mozzarella di Giovanni", Sku = "DAI-002", Price = 34.99m, Active = true },
            new Product { Name = "Gustaf's Knackebrod", Sku = "GRN-001", Price = 21.00m, Active = true },
            new Product { Name = "Tunnbrod", Sku = "GRN-002", Price = 9.00m, Active = true },
            new Product { Name = "Mishi Kobe Niku", Sku = "MEA-001", Price = 97.00m, Active = true },
            new Product { Name = "Alice Mutton", Sku = "MEA-002", Price = 39.00m, Active = true },
            new Product { Name = "Uncle Bob's Organic Dried Pears", Sku = "PRD-001", Price = 32.99m, Active = true },
            new Product { Name = "Tofu", Sku = "PRD-002", Price = 23.25m, Active = true },
            new Product { Name = "Ikura", Sku = "SEA-001", Price = 31.00m, Active = true },
            new Product { Name = "Konbu", Sku = "SEA-002", Price = 6.00m, Active = true }
        );
    }
}
```

```
        db.SaveChanges();
    }
}
```

Add `using CatalogApi.WebApi.Models.Catalog;` alongside the existing `using` block so `Category / Product` resolve. Verified end to end: `dotnet run` then `GET /api/catalog/product` returns all 16 products with the right category IDs and prices — e.g. `{ "name": "Chai", "sku": "BEV-001", "price": 18.0, "categoryId": 1 }`. `EnsureCreated()` is a demo convenience (no EF migrations required); switch to `dotnet ef database update` once the schema needs to evolve, since `EnsureCreated()` and migrations don't mix.

Importing Catalog from an existing database

If Catalog already exists as a legacy database, skip hand-typed properties entirely:

```
go run ./cmd/aspgen import-db --project ./CatalogApi `
    --script schema.sql --provider postgres --tables Products,Categories --context Catalog
```

`import-db` maps each selected table to one entity (best-effort singularized, PascalCased) through the same code path `add entity` uses. Primary keys and (on DDD-flavored backends) audit columns are skipped automatically; unmappable column types are skipped with a warning instead of failing the whole table. A `schema.sql` backup snapshot is written at the project root every run, and **no connection string is ever persisted** to the manifest, the backup, or any generated file — `dotnet ef migrations add / database update` stay a manual step you run yourself.

7. Context 2 — Inventory (`dm`): aggregates, value objects, domain services, repositories, events

`dm` is where the DDD building blocks appear: an aggregate root, immutable value objects, stateless domain services, an aggregate-scoped repository, and domain events — all as a

synchronous, headless class-library graph (Domain ← Application ← Infrastructure/Persistence, no host until a UI attaches).

```
go run ./cmd/aspngen add aggregate StockItem sku:string quantityOnHand:int reorderPoint
go run ./cmd/aspngen add value-object BinLocation aisle:string shelf:string --context I
go run ./cmd/aspngen add domain-service ReplenishmentPolicy --context Inventory --proje
go run ./cmd/aspngen add repository StockItemRepository --aggregate StockItem --context
go run ./cmd/aspngen add event StockDepletedEvent stockItemId:long quantityOnHand:int
```

The aggregate is a partial class split across two files — state/constructor and behavior — always generated together:

```
// src/NorthwindOps.DomainModel/Inventory/StockItem.cs (rendered)
public sealed partial class StockItem : BaseEntity
{
    private StockItem() { }

    public StockItem(string sku, int quantityOnHand, int reorderPoint)
    {
        Sku = DomainGuard.Required(sku, nameof(sku));
        QuantityOnHand = quantityOnHand;
        ReorderPoint = reorderPoint;
    }

    public long Id { get; private set; }
    public string Sku { get; private set; } = default!;
    public int QuantityOnHand { get; private set; }
    public int ReorderPoint { get; private set; }
    // aspngen:navigation
}
```

```
// src/NorthwindOps.DomainModel/Inventory/StockItem.Methods.cs (rendered)
public sealed partial class StockItem
{
    public void Update(string sku, int quantityOnHand, int reorderPoint)
    {
        Sku = DomainGuard.Required(sku, nameof(sku));
        QuantityOnHand = quantityOnHand;
        ReorderPoint = reorderPoint;
        Mark("system");
    }
}
```

```

    }

    public override bool IsValid() => true;
    public override void Normalize() { }
    public override void Import(BaseEntity entity) { /* ... */ }
    public override string ToString() => $"StockItem #{Id}";
}

```

`BaseEntity` (shared across every `dm` / `cqrs` aggregate — `es` -tier aggregates use a different base, `EventSourcedAggregate`, covered in Section 9) provides soft delete and audit timestamps: `Deleted`, `Created` / `LastUpdated` `TimeStamp` values, and `Init(author)` / `Mark(author)` / `SoftDelete()` / `SoftUndelete()`. Mutating operations return `CommandResponse` (`Success`, optional `Message` / `Extra`, `Ok()` / `Fail()`) instead of a bare `bool`.

The generated `CrudService` is the synchronous Application-layer entry point every `dm` aggregate gets by default:

```

// src/NorthwindOps.Application/StockItemCrudService.cs (rendered, trimmed)
public sealed class StockItemCrudService(NorthwindOpsDatabase database, IValidator<StockItem> validator)
{
    public Task<List<StockItemView>> GetAllAsync(CancellationToken cancellationToken = default)
    {
        return Task.FromResult<List<StockItemView>>(&empty);
    }

    public async Task<StockItemView> CreateAsync(StockItemRequest request, CancellationToken cancellationToken)
    {
        await validator.ValidateAndThrowAsync(request, cancellationToken);
        var entity = new StockItem(request.Sku, request.QuantityOnHand, request.ReorderPoint);
        entity.Init("system");
        database.Add(entity);
        await database.SaveChangesAsync(cancellationToken);
        return new StockItemView(entity.Id, entity.Sku, entity.QuantityOnHand, entity.ReorderPoint);
    }

    public async Task<CommandResponse> UpdateAsync(long id, StockItemRequest request, CancellationToken cancellationToken)
    {
        await validator.ValidateAndThrowAsync(request, cancellationToken);
        var entity = await database.GetByIdAsync(id, cancellationToken);
        if (entity == null) return CommandResponse.Fail("Item not found");
        entity.Update(request.QuantityOnHand, request.ReorderPoint);
        await database.SaveChangesAsync(cancellationToken);
        return CommandResponse.Ok();
    }

    public async Task<CommandResponse> DeleteAsync(long id, CancellationToken cancellationToken)
    {
        var entity = await database.GetByIdAsync(id, cancellationToken);
        if (entity == null) return CommandResponse.Fail("Item not found");
        entity.SoftDelete();
        await database.SaveChangesAsync(cancellationToken);
        return CommandResponse.Ok();
    }

    public async Task<(StockItemView[] Items, long TotalCount)> SearchAsync(string? sku, int? reorderPointMin, int? reorderPointMax, int page, int pageSize, CancellationToken cancellationToken)
    {
        var items = await database.SearchAsync(sku, reorderPointMin, reorderPointMax, page, pageSize, cancellationToken);
        return (items, items.Length);
    }
}

public sealed record StockItemRequest(string Sku, int QuantityOnHand, int ReorderPoint);
public sealed record StockItemView(long Id, string Sku, int QuantityOnHand, int ReorderPoint);

```

Use `--no-crud` on `add aggregate` when a use case should be modeled with explicit domain behavior instead of starting from generated CRUD.

`add repository` generates and wires an aggregate-scoped repository contract (Domain) plus an EF implementation (Persistence), and **patches the CrudService to actually call it** rather than leaving it dead code: `CreateAsync` becomes `repository.AddAsync(entity, ct)`, `UpdateAsync` fetches via `repository.GetByIdAsync` and saves via `repository.SaveAsync`, and `DeleteAsync` collapses to a single `repository.DeleteAsync(id, ct)`. Reads (`GetAllAsync` / `SearchAsync`) deliberately keep querying the `DbContext` directly — a CQRS-style read/write split, not a shortcut.

```
src/NorthwindOps.DomainModel/Inventory/
├─ StockItem.cs
├─ StockItem.Methods.cs
├─ BinLocation.cs                (value object, immutable record)
├─ ReplenishmentPolicy.cs        (domain service, stateless)
├─ IStockItemRepository.cs       (repository contract)
├─ StockDepletedEvent.cs         (domain event, immutable)
src/NorthwindOps.Persistence/
├─ StockItemConfiguration.cs     (IEntityTypeConfiguration<StockItem>)
├─ Repositories/StockItemRepository.cs
src/NorthwindOps.Application/
├─ StockItemCrudService.cs
├─ StockItemValidator.cs
```

Inventory has no host of its own — the WPF Desktop shell (added in Section 10) will call `StockItemCrudService` **in-process**, no HTTP involved.

8. Context 3 — Sales (`cqrs`): vertical-slice commands/queries and a WPF back office

`cqrs` keeps every `dm` building block and adds a WebApi host plus a genuine vertical-slice Application layer: one Command/Query, Handler, Validator per verb.

```
go run ./cmd/aspgen add aggregate Order number:string customer:string total:decimal p
go run ./cmd/aspgen add repository OrderRepository --aggregate Order --context Sales -
```

```
src/NorthwindOps.Application/Features/Sales/Order/
├─ CreateOrderCommand.cs
├─ CreateOrderHandler.cs
├─ UpdateOrderCommand.cs
├─ UpdateOrderHandler.cs
├─ DeleteOrderCommand.cs
├─ DeleteOrderHandler.cs
├─ GetOrderByIdQuery.cs
├─ GetOrderByIdHandler.cs
├─ GetOrdersQuery.cs
├─ GetOrdersHandler.cs
├─ SearchOrdersQuery.cs
├─ SearchOrdersHandler.cs
└─ OrderPagedResponse.cs
src/WebApi/Features/Sales/Order/
└─ OrderEndpoints.cs
```

Each Handler is a thin wrapper around the same `CrudService` the `dm` tier already generates — no duplicate repository contract is invented:

```
// Features/Sales/Order/CreateOrderCommand.cs (rendered)
namespace NorthwindOps.Application.Features.Sales.Order;

public sealed record CreateOrderCommand(string Number, string Customer, decimal Total,
```

```
// Features/Sales/Order/CreateOrderHandler.cs (rendered)
namespace NorthwindOps.Application.Features.Sales.Order;

public sealed class CreateOrderHandler(OrderCrudService service) : IHandler<CreateOrder
{
    public async Task<Result<OrderView>> HandleAsync(CreateOrderCommand request, Cance
    {
        var view = await service.CreateAsync(new OrderRequest(request.Number, request
        return Result<OrderView>.Success(view);
    }
}
```

```
}
}
```

The WebApi host is a minimal `Program.cs` that mounts every feature via a marker comment kept up to date automatically:

```
// src/WebApi/Program.cs (rendered)
using NorthwindOps.Application;
using NorthwindOps.Infrastructure;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

var app = builder.Build();
app.MapHealthChecks("/health");
// aspgen:features
app.MapGet("/", () => Results.Ok(new { application = "NorthwindOps" }));
app.Run();

public partial class Program { }
```

With both Inventory's `StockItem` and Sales' `Order` aggregates in place, attach the WPF Desktop shell:

```
go run ./cmd/aspgen add ui wpf --framework wpf --theme wpfui --theme-mode light --proj
```

`Order` 's generated `Store` calls the WebApi host over HTTP (Sales is `cqrs` -tier), while the `StockItem` store from Section 7 calls its `CrudService` in-process (Inventory is `dm` -tier) — both branches live in the same templated `{{.Name}}Store.cs.tpl`, selected by the aggregate's own tier:

```
// Desktop/Modules/Order/Services/OrderStore.cs (rendered, cqrs branch – HTTP)
public sealed class OrderStore(HttpClient http) : IOrderStore
{
    private const string Path = "/api/sales/order";
    public IReadOnlyList<OrderRow> GetAll() => http.GetFromJsonAsync<List<OrderRow>>(Path);
    public OrderRow Save(long editingId, OrderRow value)
```



```

{
    if (editingId == 0)
    {
        var response = http.PostAsJsonAsync(Path, value).GetAwaiter().GetResult();
        response.EnsureSuccessStatusCode();
        return response.Content.ReadFromJsonAsync<OrderRow>().GetAwaiter().GetResult();
    }
    http.PutAsJsonAsync($"{Path}/{editingId}", value).GetAwaiter().GetResult().EnsureSuccessStatusCode();
    return value with { Id = editingId };
}
// Delete(...), Search(...) follow the same HTTP pattern
}

```

```

// Desktop/Modules/StockItem/Services/StockItemStore.cs (rendered, dm branch – in-progress)
public sealed class StockItemStore(StockItemCrudService service) : IStockItemStore
{
    public IReadOnlyList<StockItemRow> GetAll() =>
        service.GetAllAsync().GetAwaiter().GetResult()
            .Select(x => new StockItemRow(x.Id, x.Sku, x.QuantityOnHand, x.ReorderPoint))
            .ToList();

    public StockItemRow Save(long editingId, StockItemRow value)
    {
        var request = new StockItemRequest(value.Sku, value.QuantityOnHand, value.ReorderPoint);
        if (editingId == 0)
        {
            var created = service.CreateAsync(request).GetAwaiter().GetResult();
            return new StockItemRow(created.Id, created.Sku, created.QuantityOnHand, created.ReorderPoint);
        }
        service.UpdateAsync(editingId, request).GetAwaiter().GetResult();
        return value with { Id = editingId };
    }

    public void Delete(long id) => service.DeleteAsync(id).GetAwaiter().GetResult();
}

```

`--theme wpfui` adds the WPF-UI Fluent theme (NuGet package, resource dictionaries, a themed `FluentWindow` shell); `--theme-mode light|dark` picks the starting palette (default `light`) — the shell also ships a runtime toggle, so this only affects the first paint.

9. Context 4 — Billing (`es`): event sourcing and an append-only ledger

`es` keeps everything `cqrs` has and rebuilds the aggregate around events instead of mutable state: an append-only event store with optimistic concurrency, an `EventSourcedAggregate` base class, and a synchronous read-model projection updated in the same unit of work as the event append.

`Billing` gets its own solution (per Section 4/5) since `es` 's event-sourced base class isn't compatible with the `cqrs / dm` skeleton `NorthwindOps` already has:

```
go run ./cmd/aspgen add aggregate Invoice number:string customer:string amount:decimal
```

The aggregate replays its own history instead of storing current-state columns directly:

```
// src/BillingLedger.DomainModel/Billing/Invoice.cs (rendered, trimmed)
public sealed partial class Invoice : EventSourcedAggregate
{
    private Invoice() { }

    public static Invoice LoadFromHistory(long id, IEnumerable<object> history)
    {
        var aggregate = new Invoice { Id = id };
        foreach (var domainEvent in history) aggregate.ApplyHistoric(domainEvent);
        return aggregate;
    }

    public string Number { get; private set; } = default!;
    public string Customer { get; private set; } = default!;
    public decimal Amount { get; private set; } = default!;
    public DateOnly IssuedOn { get; private set; } = default!;

    protected override void Apply(object domainEvent)
    {
        switch (domainEvent)
        {
            case InvoiceCreatedEvent created:
                Number = created.Number; Customer = created.Customer; Amount = created.Amount;
                break;
        }
    }
}
```

```

        case InvoiceUpdatedEvent updated:
            Number = updated.Number; Customer = updated.Customer; Amount = updated.Amount;
            break;
        case InvoiceDeletedEvent:
            Deleted = true;
            break;
        // aspgen:events
        default:
            throw new DomainException($"Unhandled event {domainEvent.GetType().Name}");
    }
}
}

```

The event-store repository loads by replaying, saves by appending only the newly-raised events (optimistic concurrency via an expected-version check), and immediately projects the change into a queryable read model:

```

// src/BillingLedger.Application/InvoiceEventStoreRepository.cs (rendered, trimmed)
public sealed class InvoiceEventStoreRepository(EventStore eventStore, BillingLedgerDatabase database)
{
    public async Task<Invoice?> LoadAsync(long id, CancellationToken cancellationToken)
    {
        var records = await eventStore.LoadAsync(nameof(Invoice), id, cancellationToken);
        return records.Count == 0 ? null : Invoice.LoadFromHistory(id, records.Select(r => r.Data));
    }

    public async Task SaveAsync(Invoice aggregate, CancellationToken cancellationToken)
    {
        var newEvents = aggregate.DequeueUncommittedEvents();
        if (newEvents.Count == 0) return;
        var expectedVersion = aggregate.Version - newEvents.Count;
        await eventStore.AppendAsync(nameof(Invoice), aggregate.Id, expectedVersion, newEvents);
        await ProjectAsync(aggregate, cancellationToken);
    }

    private async Task ProjectAsync(Invoice aggregate, CancellationToken cancellationToken)
    {
        var readModel = await database.InvoiceReadModels.FirstOrDefaultAsync(x => x.Id == aggregate.Id);
        // add / update / remove InvoiceReadModel to match aggregate state, then SaveChanges
    }
}

```

```
}  
}
```

`add repository` is rejected for `es`-tier aggregates — `{Aggregate}EventStoreRepository` already fills that role; there is no second contract to generate. Event versioning/schema evolution and snapshotting are deliberately out of scope for the generated code — plan for them yourself as the ledger grows.

```
src/BillingLedger.DomainModel/Billing/  
├─ Invoice.cs  
├─ Invoice.Methods.cs  
└─ InvoiceEvents.cs (Created/Updated/Deleted event records)  
src/BillingLedger.Application/  
├─ InvoiceEventStoreRepository.cs  
├─ InvoiceRequest.cs  
└─ InvoiceValidator.cs  
src/BillingLedger.Application/Features/Billing/Invoice/  
├─ CreateInvoiceCommand.cs / Handler.cs  
├─ UpdateInvoiceCommand.cs / Handler.cs  
├─ DeleteInvoiceCommand.cs / Handler.cs  
├─ GetInvoiceByIdQuery.cs / Handler.cs  
├─ GetInvoicesQuery.cs / Handler.cs  
└─ SearchInvoicesQuery.cs / Handler.cs
```

`BillingLedger` stands alone as its own deployable service — Section 10 attaches a UI to it directly (`spa` for API-first access, or `wpf` / `blazor` if a dedicated finance-desk UI is wanted; either works since `es` is the project's only, first context).

10. A tour of every UI framework

A UI attaches to the **whole project** — one UI framework surfaces every compatible aggregate in every context, which is why `NorthwindOps` above picked a single `wpf` shell spanning its `dm` + `cqrs` contexts. This section builds one small, focused project per remaining UI framework

(reusing `BillingLedger` from Section 9 for the `es` case) so each gets full coverage without overstating what one project can combine.

UI (-ui / --framework)	Compatible tiers	Transport	Notes
<code>wpf</code>	<code>dm</code> , <code>cqrs</code> , <code>es</code>	in-process (<code>dm</code>) or HTTP (<code>cqrs</code> / <code>es</code>)	Most tolerant of mixed tiers (works with a <code>dm</code> + <code>cqrs</code> combination); supports <code>--theme wpfui</code> / <code>--theme-mode</code> .
<code>blazor</code>	<code>cqrs</code> , <code>es</code>	HTTP	Blazor Server calling the WebApi host.
<code>mvc</code>	<code>dm</code> only	in-process	Classic ASP.NET Core MVC, calls <code>CrudService</code> directly — no HTTP.
<code>spa</code>	<code>cqrs</code> , <code>es</code>	HTTP (bring your own frontend)	Wires OpenAPI/Scalar + permissive local-dev CORS onto the host; scaffolds no frontend project.

Every attached UI (`wpf` / `blazor` / `mvc`) renders a full list/edit/details CRUD screen for every already-present aggregate, and keeps generating a screen automatically for every aggregate `add` ed afterward.

`blazor` — a Sales web portal (`cqrs`)

```
go run ./cmd/aspgen new SalesPortal --context Sales --arch cqrs -ui blazor --output ./
go run ./cmd/aspgen add aggregate Order number:string customer:string total:decimal p
```

The Blazor host calls the WebApi over HTTP (not in-process — a deliberate difference from the legacy Renoir Blazor profile in Section 12), and each aggregate gets a Razor CRUD page with quick search and an advanced filter panel:

```
@* Components/Pages/Sales/OrderCrud.razor (rendered, trimmed) *@
@page "/sales/orders"
@inject SalesPortal.Application.OrderCrudService Service
<h1>Orders</h1>
<EditForm Model="form" OnValidSubmit="SaveAsync">
```

```

<DataAnnotationsValidator />
<div class="mb-3"><label>Number</label><InputText @bind-Value="form.Number" class=
<div class="mb-3"><label>Customer</label><InputText @bind-Value="form.Customer" cl
<div class="mb-3"><label>Total</label><InputNumber @bind-Value="form.Total" class=
<button class="btn btn-primary" type="submit">Save</button>
</EditForm>

```

mvc — a warehouse admin dashboard (dm)

```

go run ./cmd/aspgen new WarehouseOps --context Inventory --arch dm -ui mvc --output ./
go run ./cmd/aspgen add aggregate StockItem sku:string quantityOnHand:int reorderPoint

```

MVC calls `CrudService` directly, in-process — no HTTP client, matching `dm`'s headless nature — with classic controller actions and query-string filters (no client-side state needed at all):

```

// Controllers/StockItemController.cs (rendered, trimmed)
[Route("inventory/stock-item")]
public sealed class StockItemController(StockItemCrudService service) : Controller
{
    private const int PageSize = 25;

    [HttpGet("")]
    public async Task<IActionResult> Index(string? search, int? quantityOnHandMin, int
    {
        if (page < 1) page = 1;
        var (items, totalCount) = await service.SearchAsync(search, quantityOnHandMin,
        return View(new StockItemIndexViewModel(items, totalCount, page, PageSize, se
    }

    [HttpPost("create")]
    public async Task<IActionResult> Create(StockItemRequest request, CancellationToke
    {
        try { await service.CreateAsync(request, cancellationToken); return RedirectTo
        catch (ValidationException ex) { foreach (var f in ex.Errors) ModelState.AddMc
    }
    // Details, Edit, Delete follow the same pattern
}

```

spa — an API-first Billing service (es)

`BillingLedger` (from Section 9) already has its `Invoice` aggregate; attach `spa` to it directly instead of scaffolding a new project:

```
go run ./cmd/aspgen add ui spa --framework spa --project ./BillingLedger
```

`spa` scaffolds no frontend project at all — it patches `Program.cs` / `WebApi.csproj` in place with OpenAPI discovery, Scalar (`/scalar/v1`), and a permissive local-dev CORS policy, so any hand-written or separately generated frontend (React, Vue, Angular, a mobile app) can call the API immediately during development.

Every UI in this guide can be attached either at `new` time (`-ui`) or afterward with `add ui --framework` :

```
go run ./cmd/aspgen add ui spa --framework spa --project ./BillingLedger
go run ./cmd/aspgen add ui wpf --framework wpf --project ./NorthwindOps
go run ./cmd/aspgen add ui blazor --framework blazor --project ./SalesPortal
go run ./cmd/aspgen add ui mvc --framework mvc --project ./WarehouseOps
```

11. Extending every layer — recipes for real feature work

Domain layer

- **Add a property to an existing aggregate/entity** without hand-editing every generated layer:

```
powershell go run ./cmd/aspgen add entity-field StockItem binLocation:string --project
./NorthwindOps
```

`entity-field` patches the already-rendered Domain class, persistence configuration, Application request/response records, Handlers/Validators/Endpoints, seed data, and every attached UI layer (WPF Row/View/ViewModel, Blazor form model, MVC view model) in place — no regeneration, no duplication.

- **Many-to-one relations:** `category:Category` (required) or `category:Category?` (optional) inside any `add entity` / `add aggregate` call, synthesizing the FK property and navigation.
- **Many-to-many relations:** `tags:Tag[]` synthesizes a join aggregate/entity (e.g. `ProductTag`) in the same context, with two required many-to-one relations back to both sides — no hand-written join table needed.

Application layer

- `cqrs` / `es` Handlers are intentionally thin — put orchestration logic in the Handler, not the Endpoint; keep the Endpoint a pure HTTP-shape adapter.
- `dm` 's `CrudService` is the natural place for synchronous cross-cutting business rules that don't need the vertical-slice ceremony yet — promote to `cqrs` when the API surface earns it.

Infrastructure / Persistence

- **Switch or add a database provider:**

```
powershell go run ./cmd/aspngen add database postgres --project ./NorthwindOps
```

`sqlite` is the default everywhere; `postgres` is recorded in `.aspngen/manifest.json` and emitted into the EF Core provider registration, connection string, and DI setup. `dm` -tier contexts are class libraries with no host yet to attach a provider to — the provider takes effect once a UI or a `cqrs` / `es` host exists.

- **Import more tables later:** `import-db` is incremental too — rerun it with a fresh `--script` / `--connection` and `--tables` to add more entities/aggregates without touching what's already generated.

WebApi layer

- Feature endpoints for `cqrs` / `es` register themselves via the `// aspngen:features` marker in `Program.cs` — never hand-edit that block; let `add aggregate` / `add repository` own it.
- `/health` , `/openapi/v1.json` , and `/scalar/v1` are wired on every host tier that has one (`ar` / `cqrs` / `es`).

UI layer

- `add module NAME` adds a new Prism module shell to an existing WPF project (run `add ui wpf` first if the project has none yet) — legacy `--app / --backend` profile only.
- `add ui` is idempotent per framework: re-running it against a project that already has aggregates renders any screens that are still missing (e.g. after `add aggregate` added something new) without touching existing ones.

Tests and CI

Every context/arch project gets `tests\{Project}.UnitTests` (an in-memory `DbContext` smoke test) and, for any tier with a host (`ar / cqrs / es`), `tests\{Project}.IntegrationTests` (`WebApplicationFactory<Program>` hitting the root endpoint). `scripts\ci.ps1` drives restore/build/test and optionally publish:

```
.\scripts\ci.ps1           # restore, build, test
.\scripts\ci.ps1 -Publish   # ...and publish the WebApi host (skipped for headless c
.\scripts\ci.ps1 -SkipTests -Configuration Debug
```

12. The legacy `--app / --backend` workflow, including the Renoir DDD Blazor profile

The original workflow predates `--context / --arch` and remains fully supported. It's still the right choice if your team already standardized on it, or you specifically want the Renoir-style Blazor DDD profile (which has no equivalent in the newer engine).

Profile matrix

<code>webapi</code>	Clean Architecture Web API
<code>webapi --backend ddd</code>	Clean Architecture + DDD/CQRS CRUD
<code>webapi --simple</code>	Single-project Active Record-style CRUD API
<code>wpf</code>	Prism/DryIoc desktop shell and local UI modules
<code>wpf --backend ddd</code>	Local DDD + SQLite layers with no WebApi

<code>fullstack --backend ddd</code>	DDD/CQRS API + Prism/DryIoc WPF modules
<code>fullstack --simple</code>	Simple CRUD API + WPF modules connected by HttpClient
<code>blazor</code>	Renoir-style DDD Blazor profile (see below)

`--simple` and `--backend ddd` are mutually exclusive. `--theme wpfui` (plus `--theme-mode light|dark`) applies to any `wpf` / `fullstack` project. `--seed dummy [N]` (default 3 records per entity) works with `--simple` or `--backend ddd`, seeding at API/WPF startup for development environments only.

Worked example: RenoirCommerce (`--app blazor`)

```
go run ./cmd/aspgen new RenoirCommerce --app blazor --output ./RenoirCommerce
go run ./cmd/aspgen add context Catalog --project ./RenoirCommerce
go run ./cmd/aspgen add aggregate Product name:string price:decimal active:bool public:bool
go run ./cmd/aspgen add value-object ProductCode value:string --context Catalog --project ./RenoirCommerce
go run ./cmd/aspgen add domain-service PricingPolicy --context Catalog --project ./RenoirCommerce
go run ./cmd/aspgen add repository ProductRepository --aggregate Product --context Catalog --project ./RenoirCommerce
go run ./cmd/aspgen add event ProductPriceChanged productId:long price:decimal --context Catalog --project ./RenoirCommerce
```

The Renoir profile's structural conventions are modeled directly on a real reference implementation (the Renoir sample from Dino Esposito's *Clean Architecture with .NET*), kept async instead of the reference's synchronous style — the same `BaseEntity` / `CommandResponse` /partial-class-aggregate conventions used by the `dm` + engine tiers above actually originate here. The generated Blazor CRUD page renders in-process (no HTTP client, unlike the newer `blazor-context` UI in Section 10) and never binds directly to the domain aggregate — it uses a local editable form model plus the same immutable `Request` / `View` records:

```
@* Components/Pages/Catalog/ProductCrud.razor (rendered, trimmed) *@
@page "/catalog/products"
@inject RenoirCommerce.Application.ProductCrudService Service
<h1>Products</h1>
<EditForm Model="form" OnValidSubmit="SaveAsync">
    <DataAnnotationsValidator />
    <div class="mb-3"><label>Name</label><InputText @bind-Value="form.Name" class="form-control"/>
    <div class="mb-3"><label>Price</label><InputNumber @bind-Value="form.Price" class="form-control"/>
```

```

<div class="mb-3"><label>Active</label><InputCheckbox @bind-Value="form.Active" c
<div class="mb-3"><label>Published</label><InputDate @bind-Value="form.Published"
<button class="btn btn-primary" type="submit">Save</button>
</EditForm>
<div class="mb-3 d-flex gap-2">
    <input @bind="searchText" @bind:event="oninput" class="form-control" placeholder="
    <button class="btn btn-outline-primary" @onclick="SearchAsync">Search</button>
</div>

```

Controls are chosen by declared type: `string` → `InputText`, `numeric` → `InputNumber`, `bool` → `InputCheckbox`, `date / datetime` → `InputDate`. Use `--no-crud` on `add aggregate` when a use case should be modeled explicitly instead of starting from generated CRUD.

Worked example: a Rails-style simple CRUD API + WPF fullstack

```

go run ./cmd/aspgen new MyApp --app fullstack --simple --theme wpfui --theme-mode dark
go run ./cmd/aspgen add entity Customer name:string age:int active:bool --project ./My
go run ./cmd/aspgen add module Customers --project ./MyApp
go run ./cmd/aspgen add database postgres --project ./MyApp
go run ./cmd/aspgen add service Email --project ./MyApp
go run ./cmd/aspgen add feature CreateCustomer name:string age:int active:bool --proje

```

This is one project, one Web API (Active Record-style, no DDD/CQRS layer), and Prism/Dryloc WPF modules connected over HTTP (`ASPGENT_API_URL`, default `http://localhost:5000`). `add feature` is the base Clean Architecture profile's vertical-slice add (request/response records, FluentValidation validator, CQRS handler returning `Result<T>`, Minimal API endpoint) — distinct from `--simple`'s flat entity CRUD and from the newer engine's `cqrs`-tier feature generation, but structurally similar.

Legacy `import-db`

```

go run ./cmd/aspgen new MyApp --app webapi --simple --script schema.sql --provider pos
go run ./cmd/aspgen import-db --project ./MyApp --connection "file:demo.db" --provider
go run ./cmd/aspgen new RenoirCommerce --app blazor --script schema.sql --provider sql

```

Supported providers: `sqlite`, `postgres`, `sqlserver`, `mysql`. `--connection` and `--script` are mutually exclusive and both require `--provider`. On the `blazor` profile, `--context` is required since tables become aggregates, not flat entities.

13. Custom templates

Every rendered file comes from an embedded Go `text/template` tree. Export, edit, and point aspgen at your own copy:

```
go run ./cmd/aspgen templates export ./my-templates
go run ./cmd/aspgen templates list
go run ./cmd/aspgen templates validate ./my-templates
go run ./cmd/aspgen new MyApp --app webapi --templates ./my-templates
```

`--templates PATH` is also accepted by `new` in the context/arch engine. Validate a customized tree before pointing real generation at it — `templates validate` parses every template without rendering, catching syntax errors early.

14. Testing, CI, and production readiness

Verification checklist before shipping a change built on aspgen-generated code (or before trusting a generated project in CI):

1. `go build / go vet / go test` the generator itself if you're customizing templates (`./cmd/... ./internal/...`, never bare `./...` — example assets under `agent/skills/**` don't compile standalone).
2. `templates validate` any custom template directory.
3. `dotnet restore / build / test` the generated solution — `scripts\ci.ps1` does this in one call and mirrors what CI runs.
4. Run generation twice against the same output and confirm no unwanted duplication (idempotency is a design goal, not an afterthought).

5. Review project references match the intended dependency direction: Domain ← Application ← Infrastructure ← WebApi (and Desktop → Infrastructure/Application/Domain for `wpf --backend ddd`).
6. Confirm `.aspgen/manifest.json` accurately reflects every context/aggregate/entity/UI — future `add` commands trust it completely.
7. For production, review `appsettings.json` connection strings yourself; aspgen never writes production secrets and never runs `dotnet ef` — migrations stay a manual, reviewed step.

15. Full flag reference

`aspgen new` — **context/arch engine**

Flag	Values	Notes
<code>--context CTX</code>	any name	Presence routes into the engine instead of <code>--app / --backend</code> .
<code>--arch TIER</code>	<code>ar</code> , <code>dm</code> , <code>cqrs</code> , <code>es</code>	All four tiers implemented.
<code>-ui UI</code>	<code>wpf</code> , <code>blazor</code> , <code>spa</code> , <code>mvc</code>	See Section 10 for tier compatibility; omit for a headless project.
<code>--database DB</code>	<code>sqlite</code> (default), <code>postgres</code>	<code>dm</code> -tier has no host to attach the provider to yet.
<code>--output PATH</code>	directory	Default: project name.
<code>--templates PATH</code>	directory	Use a custom template set instead of the embedded one.
<code>--no-tests</code>	flag	Skip <code>{Project}.UnitTests</code> / <code>{Project}.IntegrationTests</code> .

Flag	Values	Notes
<code>--dry-run</code>	flag	Print planned changes without writing files.
<code>--force</code>	flag	Overwrite existing files.

`aspngen new` — **legacy** `--app` / `--backend`

Flag	Values	Notes
<code>--app</code> TARGET	<code>webapi</code> (default), <code>wpf</code> , <code>blazor</code> , <code>fullstack</code>	
<code>--simple</code>	flag	Rails-style Active Record CRUD; not with <code>--backend ddd</code> .
<code>--backend</code> ddd	flag	Clean Architecture DDD/CQRS layers (<code>webapi</code> / <code>fullstack</code> , or <code>wpf</code> for local-only DDD).
<code>--database</code> DB	<code>sqlite</code> (default), <code>postgres</code>	<code>webapi</code> / <code>fullstack</code> / <code>wpf+ddd</code> only.
<code>--theme</code> wpfui	flag	WPF-UI Fluent theme (<code>wpf</code> / <code>fullstack</code> only).
<code>--theme-</code> mode MODE	<code>light</code> (default), <code>dark</code>	Requires <code>--theme wpfui</code> .
<code>--seed</code> dummy [N]	count, default 3	Requires <code>--simple</code> or <code>--backend ddd</code> .
<code>--script</code> PATH	file	SQL DDL to import entities/aggregates from; requires <code>-provider</code> .
<code>--provider</code> P	<code>sqlite</code> , <code>postgres</code> , <code>sqlserver</code> , <code>mysql</code>	Required with <code>--script</code> .
<code>--tables</code> T	<code>all</code> (default) or comma list	

Flag	Values	Notes
<code>--context CTX</code>	name	Required with <code>--script</code> on the <code>blazor</code> profile (tables become aggregates).

aspngen add — kinds

Kind	Usage	Profile
<code>entity</code>	<code>add entity NAME prop:type... [-context CTX]</code>	Simple/legacy, or <code>ar</code> -tier engine context with <code>--context</code> .
<code>entity-field</code>	<code>add entity-field NAME prop:type...</code>	Adds properties to an existing entity/aggregate, any profile.
<code>module</code>	<code>add module NAME</code>	WPF Prism module; legacy <code>--app / --backend</code> , run <code>add ui</code> first.
<code>database</code>	<code>add database NAME</code>	Register/switch the persistence provider.
<code>service</code>	<code>add service NAME</code>	Application service; legacy non-simple <code>webapi / fullstack</code> .
<code>feature</code>	<code>add feature NAME prop:type...</code>	Web API vertical-slice CRUD feature; legacy Clean Architecture profile.
<code>ui</code>	<code>add ui --framework wpf\ blazor\ spa\ mvc</code>	Legacy: WPF onto a webapi project. Engine: attach to a <code>--context / --arch</code> project.
<code>context</code>	<code>add context NAME [--arch TIER]</code>	Blazor/Renoir profile without <code>--arch</code> ; engine context with <code>--arch</code> .
<code>aggregate</code>	<code>add aggregate NAME prop:type... --context CTX [--no-crud]</code>	Blazor/Renoir profile, or <code>dm</code> + engine context.
<code>value-object</code>	<code>add value-object NAME prop:type... --context CTX</code>	Blazor/Renoir profile, or <code>dm</code> + engine context.

Kind	Usage	Profile
domain-service	add domain-service NAME --context CTX	Blazor/Renoir profile, or dm + engine context.
repository	add repository NAME --aggregate AGG --context CTX	Blazor/Renoir profile, or dm + engine context (not es).
event	add event NAME prop:type... --context CTX	Blazor/Renoir profile, or dm + engine context.

Common add flags: --project PATH (default: search up from cwd for .aspgen/manifest.json), --theme wpfui / --theme-mode MODE (for ui / module), --dry-run , --force .

aspgen import-db

Flag	Values	Notes
--project PATH	directory	Default: search up from cwd.
--script PATH	file	SQL DDL to parse (mutually exclusive with --connection).
--connection STR	connection string	Live introspection (mutually exclusive with --script).
--provider P	sqlite , postgres , sqlserver , mysql	Required.
--tables T	all (default) or comma list	
--backend ddd	flag	Override the project's own backend profile.
--context CTX	name	Required for the Blazor/Renoir profile (tables → aggregates).
--dry-run / --force	flag	Same semantics as new / add .

Property type syntax

`name:type` pairs, nullable with a trailing `?` (e.g. `middleName:string?`). Accepted types: `string`, `int`, `long`, `decimal`, `float`, `bool`, `date` ($\rightarrow$ `DateOnly`), `datetime` ($\rightarrow$ `DateTime`), `guid` / `uuid` ($\rightarrow$ `Guid`). Relations: `nav:Entity` (required many-to-one), `nav:Entity?` (optional many-to-one), `nav:Entity[]` (many-to-many, synthesizes a join entity/aggregate). Unknown types are always a hard error.

16. Troubleshooting and known gotchas

- `add` fails with "no manifest found" — you're outside the generated project tree and its parents; pass `--project PATH` explicitly.
- `spa / blazor / mvc` UI rejected on a context — check the tier compatibility table in Section 10; `spa / blazor` need `cqrs / es`, `mvc` needs `dm`, and `ar`-tier contexts get no generated UI screens at all by design.
- `add repository` rejected on an `es` aggregate — expected; the aggregate already has a generated `{Aggregate}EventStoreRepository`.
- `--theme-mode` rejected without `--theme wpfui` — the flag only has meaning alongside the WPF-UI Fluent theme.
- **Property type rejected** — only the type list in Section 15's property syntax table is supported; anything else is a deliberate hard error rather than a silent pass-through.
- **A generated entity is named something like `Task`** — avoid names that collide with common .NET/BCL types (`System.Threading.Tasks.Task` is the classic one); it compiles as ambiguous C#. Use a more specific domain name instead (`TodoItem`, not `Task`).
- `go test -race` on generated Go tooling changes fails locally with "requires cgo" — a local-machine toolchain gap, not a generator bug; CI runners have cgo enabled.
- `add context --arch es / add aggregate` fails with a missing type like `EventSourcedAggregate`, or `add context --arch ar` fails with "no owning .csproj found" — you mixed a tier into a solution whose first context can't support it (see the Section 4 callout). Bootstrap that tier's context first instead, or give it its own solution: `dm` contexts

are safe to add on top of a `cqrs` -first (or `dm` -first) project; `es` and `ar` contexts should each get their own solution.

- `dotnet restore` **fails with NU1301 against a corporate NuGet feed** — restore explicitly from `nuget.org` (`dotnet restore <project> -s https://api.nuget.org/v3/index.json`), then `dotnet build` normally.

17. Next steps

- Grow Northwind Trading incrementally: add `add aggregate ShipmentRoute ...` to Inventory, `add feature ApplyDiscount ...` under Sales, or a new `Returns` context at whichever tier its complexity actually earns.
- Point `templates export` at a copy of the embedded set and adapt naming/status-code/validation conventions to your team's own standards before scaling this to a second real product.
- Wire `scripts\ci.ps1` into your existing CI pipeline (`-Publish` on merge to main, plain restore/build/test on pull requests) so every generated context stays honestly buildable, not just generatable.